

Ganatri Enhancement Report

Security, quality, and performance findings across **470 passing tests** — with a prioritised action plan.

Executive Summary. The Ganatri codebase is well-structured with a sound server-authoritative architecture and 470 passing tests. However, this review uncovered **5 critical** and **13 major** issues spanning security vulnerabilities, reliability bugs, and performance gaps that must be resolved before public deployment. The most urgent findings are a missing room-membership check in WebRTC signaling, an OAuth token exposed in a URL query string, and an admin-secret configuration that defaults to open access. A prioritised top-10 action plan closes this report.

SECTION 1 — SECURITY

Security Vulnerabilities

S-1 — Voice relay has no room-membership check **CRITICAL**

All four WebRTC signaling handlers (`VOICE_OFFER` , `VOICE_ANSWER` , `VOICE_ICE` , `VOICE_RENEGOTIATE`) accept a `targetPlayerId` from the client and forward the payload to that player with no check that sender and receiver share a room. Any authenticated socket can send a WebRTC offer to any other player on the server.

File: `packages/server/src/handlers.ts` lines 1062-1102

Recommendation: Look up both sessions, compare `session.roomCode` with `target.roomCode` , and reject if they differ or if either is null.

S-2 — OAuth `auth_token` exposed in URL query parameter **CRITICAL**

File: `packages/server/src/createApp.ts` line 321

After Google OAuth callback the server issues `redirectUrl.searchParams.set('auth_token', token)` . The bearer token appears in the browser URL bar, browser history, and `Referer` headers on subsequent navigations, and is captured by any server access-log.

Recommendation: Use `window.location.hash` (fragment), `sessionStorage` set by a relay page, or a short-lived one-time code exchanged via POST rather than a full auth token in the query string.

S-3 — `ADMIN_SECRET` not set means any secret is accepted CRITICAL

File: `packages/server/src/config.ts` line 105

`isValidAdminSecret` returns `true` unconditionally when `ADMIN_SECRET` is not set. Any logged-in user can elevate to admin on a misconfigured production server.

Recommendation: Fail-closed: if `ADMIN_SECRET` is not set, log a startup warning and return `false` (deny all). Require the env var in production.

S-4 — `sanitizePlayerName` only strips `<>` — injection risk MAJOR

File: `packages/server/src/handlers.ts` lines 130–139

The sanitizer removes `<` and `>` but passes `"`, `'`, backtick, `&`, and `\` unchanged. If player names are ever interpolated into HTML, SQL strings, or log messages, these characters can cause injection.

Recommendation: Adopt an allowlist — keep only printable Unicode letters, digits, spaces, and common punctuation (e.g., `-_.!?`).

S-5 — `readJsonBody` has no size limit MAJOR

File: `packages/server/src/createApp.ts` lines 90–97

All POST body bytes are buffered into memory before parsing with no byte counter or `Content-Length` check. A client can POST a very large body to any HTTP endpoint and exhaust server memory.

Recommendation: Add a running byte counter and return `413` if it exceeds 64 KB.

S-6 — `Socket.io` has no `maxHttpBufferSize` set MAJOR

File: `packages/server/src/createApp.ts` lines 367–375

`Socket.io` defaults to 1 MB buffer. For a card-game server where the largest event payload is a few kilobytes, 1 MB is 100x too permissive. Any client can send a ~1 MB event and consume that memory per-event.

Recommendation: Set `maxHttpBufferSize: 64 * 1024` in the `new Server(...)` options.

S-7 — Auth token written to `localStorage` after OAuth login MAJOR

File: `packages/web/src/net/socket.ts` lines 72–79

`captureAuthTokenFromUrl` writes the OAuth token to `localStorage` (line 77) in addition to `sessionStorage`. `localStorage` is persistent across browser sessions and available to any JS on the same origin indefinitely.

Recommendation: Remove the `localStorage.setItem(LEGACY_TOKEN_KEY, authToken)` call. `sessionStorage` (already set at line 76) is sufficient.

S-8 — Ghost-session adoption trusts client-supplied `playerId` LOW

File: `packages/server/src/handlers.ts` line 499

On server restart, the code re-adopts a session by the `playerId` sent in the socket auth handshake. The adoption check relies on the target session having no active socket, but this is not explicitly enforced server-side.

Recommendation: Verify that `tryAdoptGhostSession` confirms `socketId === null` before allowing adoption. Add a unit test for this path.

SECTION 2 — CODE QUALITY & REFACTORING

Code Quality Issues

Q-1 — `maxPlayers` admin config accepts 5-8 but engine rejects >4 CRITICAL

File: `packages/server/src/handlers.ts` line 972

`isValidPlayers` allows values 2-8. The engine's `createGame` enforces a hard maximum of 4 players and throws on violation. If an admin sets `maxPlayers = 5` and five players join and start, the server throws an uncaught error and the room is left in an inconsistent state.

Recommendation: Change the upper bound to 4 in the server-side validator, or propagate the engine constraint into a shared constant and reference it from both places.

Q-2 — `emitAck` never rejects and has no timeout CRITICAL

File: `packages/web/src/net/socket.ts` lines 179-187

`emitAck<T>` wraps `socket.emit` in a Promise that only resolves, never rejects, with no timeout. If the server disconnects mid-emit, the Promise hangs indefinitely. UI elements holding a `submitting` flag freeze with no recovery path.

Recommendation: Race against a 10-second timeout and reject with a typed error. Propagate the error to the UI so the user can retry.

Q-3 — `silentLeaveRoom` and `removeFromLobby` duplicate logic MAJOR

File: `packages/server/src/handlers.ts` lines 1218-1292

Both functions implement "remove player from lobby, reassign host, delete empty room, broadcast update". The two paths can diverge: `silentLeaveRoom` (line 1243) does not check `session.roomCode !== roomCode` before acting, while `removeFromLobby` (line 1221) does.

Recommendation: Refactor so `silentLeaveRoom` calls `removeFromLobby` for the LOBBY branch.

Q-4 — `updateSession` does not cover `userId` or `account` fields MAJOR

File: `packages/server/src/store.ts` lines 132–138

`updateSession` only applies patches to `name`, `roomCode`, and `socketId`. The `userId` and `account` fields are mutated directly throughout `handlers.ts`, bypassing the canonical mutation function and making cross-cutting concerns harder to add.

Recommendation: Add `userId` and `account` (and `isGuest`) to the fields managed by `updateSession`.

Q-5 — `handleStartGame` stale Maps on "play again" MAJOR

File: `packages/server/src/handlers.ts` lines 1556–1563

When restarting from `DONE` phase, `room.disconnectedAt` (a Map) and `room.gracePeriodTimers` (a Map) are not cleared. Stale grace-period timers from the previous game can fire against the new game's state.

Recommendation: Add `room.disconnectedAt = new Map();`
`room.gracePeriodTimers.forEach(clearTimeout);` `room.gracePeriodTimers = new Map();`
before creating the new game.

Q-6 — `isMakeMovePayload` does not cap the capture array length MAJOR

File: `packages/server/src/handlers.ts` lines 2508–2510

The validator checks that `capture` is an array of strings but does not bound its length. A client can submit thousands of card IDs; the engine iterates over all of them before returning an error.

Recommendation: Add `if ((m['capture'] as unknown[]).length > 52) return false;`.

Q-7 — `isJoinRoomPayload` has no length or format validation on `roomCode`

MAJOR

File: `packages/server/src/handlers.ts` lines 2495–2496

Only `typeof roomCode === 'string'` is checked. An arbitrarily long string reaches `getRoom(roomCode)` without format validation.

Recommendation: Validate that `roomCode` matches `/^[A-Z0-9]{6}$/` with a length cap of 16 characters.

Q-8 — `generateRoomCode` uses `Math.random()` LOW

File: `packages/server/src/handlers.ts` lines 2480–2488

Room codes use `Math.random()` which is not a cryptographically secure RNG. If room codes ever need to be unpredictable to prevent enumeration attacks, this would be a problem.

Recommendation: Switch to `crypto.randomInt` for robustness.

Q-9 — No rate limiting on most socket events MAJOR

File: `packages/server/src/handlers.ts` (global)

`make_move` has a throttle guard but `create_room`, `join_room`, `start_game`, `request_state`, `invite_player`, etc. have no rate limiting. A bot can spam the server freely.

Recommendation: Add a simple token-bucket or sliding-window rate limiter per `socket.id` at the `handleConnection` level.

Q-10 — `handleStartGame` seed is `Date.now()` — may collide LOW

File: `packages/server/src/handlers.ts` line 1560

The game seed uses `Date.now()` which can collide if two games start in the same millisecond, reducing true reproducibility in logs.

Recommendation: Use a monotonically incrementing counter or a cryptographically random 32-bit integer as the seed.

Q-11 — `setScreen` missing from `useMemo` dependency array LOW

File: `packages/web/src/state/GameProvider.tsx` lines 529-577

`setScreen` is included in the context value but absent from the dependency array — a lint violation. Since `setScreen` is a `useState` setter (stable), this doesn't currently cause a bug.

Recommendation: Add `setScreen` to the dependency array or use a functional-update form.

Q-12 — `emitAck` type parameter is unchecked at runtime LOW

File: `packages/web/src/net/socket.ts` lines 179-187

The generic `emitAck<T>` casts the ack callback argument as `T` without runtime validation. If the server sends an unexpected shape, the client uses malformed data silently.

Recommendation: Pass an optional structural-check or Zod schema to `emitAck` and reject if validation fails.

SECTION 3 — MISSING FEATURES / UX GAPS

UX & Feature Gaps

U-1 — `HANDS_REDISTRIBUTED` event has no UI representation MAJOR

File: `packages/web/src/screens/GameScreen.tsx` lines 44-53

`flashFor` handles `TRICK_WON` and `PLAYER_SAFE` but `HANDS_REDISTRIBUTED` (the stalemate resolution event) produces no visible flash or notification. Players see their hands change silently with no explanation.

Recommendation: Add a `HANDS_REDISTRIBUTED` case to `flashFor` returning something like `{ kind: 'won', text: 'Stalemate – hands redistributed!' }`.

U-2 — `legalPart2CardIds` re-implements engine logic on the client MAJOR

File: `packages/web/src/game/legal.ts` lines 1-28

The client-side legality helper re-implements the Part 2 follow-suit rule rather than importing from the engine. If engine follow-suit logic changes, client card highlighting diverges silently.

Recommendation: Export a view-only version of `legalPart2Cards` from `packages/engine` accepting `PlayerView`, and use it in both places.

U-3 — `Part1Board` confirmation hint misleadingly says "final" LOW

File: `packages/web/src/components/Part1Board.tsx` line 155

The hint reads "Are you sure? This choice is final." but a cancel button is available. "Final" is misleading to players.

Recommendation: Change to "Confirm your card choice. You can still change your mind."

U-4 — `progressionLoading` / `progressionError` state has no UI feedback LOW

File: `packages/web/src/state/GameProvider.tsx` (context)

The context exposes `progressionLoading` and `progressionError` but these are not observed for visible UI feedback. Users waiting for progression data after login see no indicator.

Recommendation: Verify that progression screens use `progressionLoading` for a spinner and `progressionError` for a retry prompt.

U-5 — `applyAutoMove` always picks `moves[0]` with no documentation LOW

File: `packages/server/src/handlers.ts` lines 250-311

When a player's turn times out the server plays the first legal move silently. This is undocumented and may surprise players expecting randomness or a smarter fallback.

Recommendation: Add a comment documenting the deterministic pick, and consider surfacing this to the player ("Your turn timed out; a card was played automatically").

SECTION 4 — PERFORMANCE

Performance Findings

P-1 — `matchScoringForState` computed N+1 times per move MAJOR

File: `packages/server/src/handlers.ts` lines 1646-1662

After every `make_move`, `matchScoringForState` is called once per non-mover player in the send loop and then again for the mover's ack — up to 4 redundant calls per move. Each call iterates over the full event log.

Recommendation: Call `matchScoringForState` once, store the result, and reuse it for all players and the ack.

P-2 — `GameProvider` `useMemo` with 30+ dependencies rebuilds on every game event MAJOR

File: `packages/web/src/state/GameProvider.tsx` lines 478–578

The context value memo has ~30 dependencies including `lastEvent`. `lastEvent` updates on every game event (every card played, every trick resolved), rebuilding the entire context and re-rendering every consumer.

Recommendation: Split the context into stable sub-contexts (auth, lobby, game state, events). Components subscribe only to the slice they need, or move `lastEvent` to a dedicated hook backed by a ref + force-update.

P-3 — In-memory session store has no eviction of stale sessions MAJOR

File: `packages/server/src/store.ts` (entire file)

Sessions are created on every new connection but only removed on `deleteAccount`. A server running for weeks accumulates sessions for every guest who ever connected — a memory leak proportional to total unique connections.

Recommendation: Run a periodic cleanup (e.g., every 30 minutes) removing sessions where `socketId` is null, `roomCode` is null, `userId` is null (guest), and `lastSeenAt` is older than a TTL (e.g., 48 hours).

PRIORITISED ACTION PLAN

Top-10 Actions Before Public Deployment

1 Fix voice relay room-membership check

Validate that sender and target share a room before forwarding any WebRTC signal.

`handlers.ts:1066`

Critical — Security

2 Fix OAuth `auth_token` in redirect URL

Move token delivery out of the URL query string to prevent history and referrer leaks.

`createApp.ts:321`

Critical — Security

3 Fail-close `isValidAdminSecret` when env var not set

Return `false` and log a startup error requiring the env var to be set before the server accepts admin requests. `config.ts:105`

Critical — Security

4 Cap `maxPlayers` admin config at 4

Align the server validator with the engine's hard limit of 4 players to prevent uncaught errors on game start. `handlers.ts:972`

Critical — Correctness

5 Add timeout to `emitAck`

Race all socket acks against a 10-second timer; reject the promise and unblock the UI on timeout. `socket.ts:179`

Critical — Reliability

6 Remove auth token write to `localStorage`

Eliminate the longer-lived copy of the OAuth session token; `sessionStorage` is already set. `socket.ts:77`

Major — Security

7 Add capture array length cap in `isMakeMovePayload`

Trivial one-liner preventing engineered CPU spikes from oversized capture arrays: reject if `length > 52`. `handlers.ts:2509`

Major — Security / DoS

8 Add HTTP body size limit to `readJsonBody`

Return `413` after 64 KB to prevent memory exhaustion on large POST payloads. `createApp.ts:91`

Major — Security / DoS

9 Fix stale Maps on "play again"

Clear `disconnectedAt` and cancel all `gracePeriodTimers` before creating a new game to prevent stale timers firing against new game state. `handlers.ts:1557`

Major — Correctness

10 Compute `matchScoringForState` once per move

Hoist the call above the per-player send loop and reuse the result for all players and the ack, eliminating up to 4 redundant full-log iterations per move. `handlers.ts:1648–1661`

Major — Performance

→ Severity key: **Critical** = blocking for production · **Major** = must fix before GA · **Low** = improve when convenient. All 470 existing tests pass; none of the findings above break current tests, but several require new test coverage once fixed.